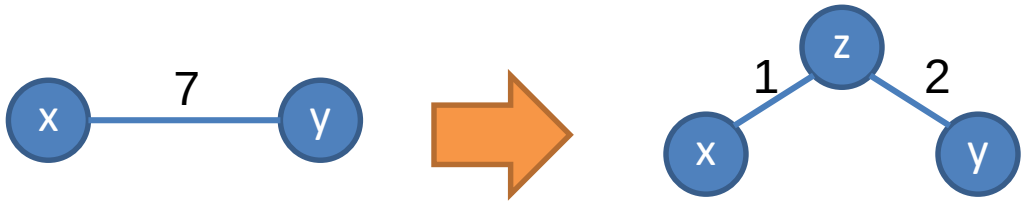
Shortest Path3

Floyd–Warshall algorithm

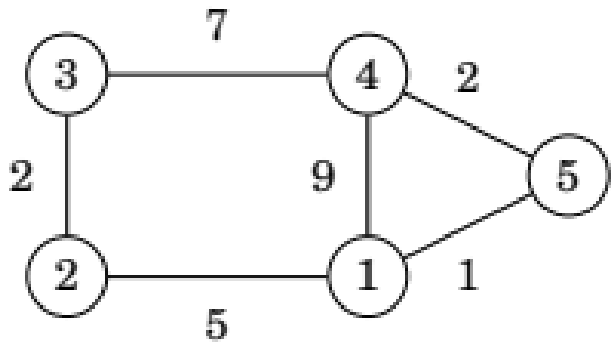
Floyd-Warshall algorithm

- Floyd-Warshall algorithm เป็นทางเลือกในการหา shortest path
- ไม่เหมือนกับอัลกอริทึมที่ผ่านมา **มันจะหาทุก shortest path ระหว่างทุกคู่โหนด (All pair shortest path)**
- อัลกอริทึมนี้เก็บ **array 2 มิติ**ที่เป็น**ระยะทางระหว่างโหนด**
- เริ่มต้นระยะทางจะถูกคำนวณโดยใช้<u>เส้นเชื่อมที่เชื่อมโดยตรง</u>ระหว่างโหนด
- หลังจากนั้นอัลกอริทึมจะลดระยะทางโดยใช้ intermediate node ใน path



การเก็บข้อมูล

- พิจารณากราฟต่อไปนี้

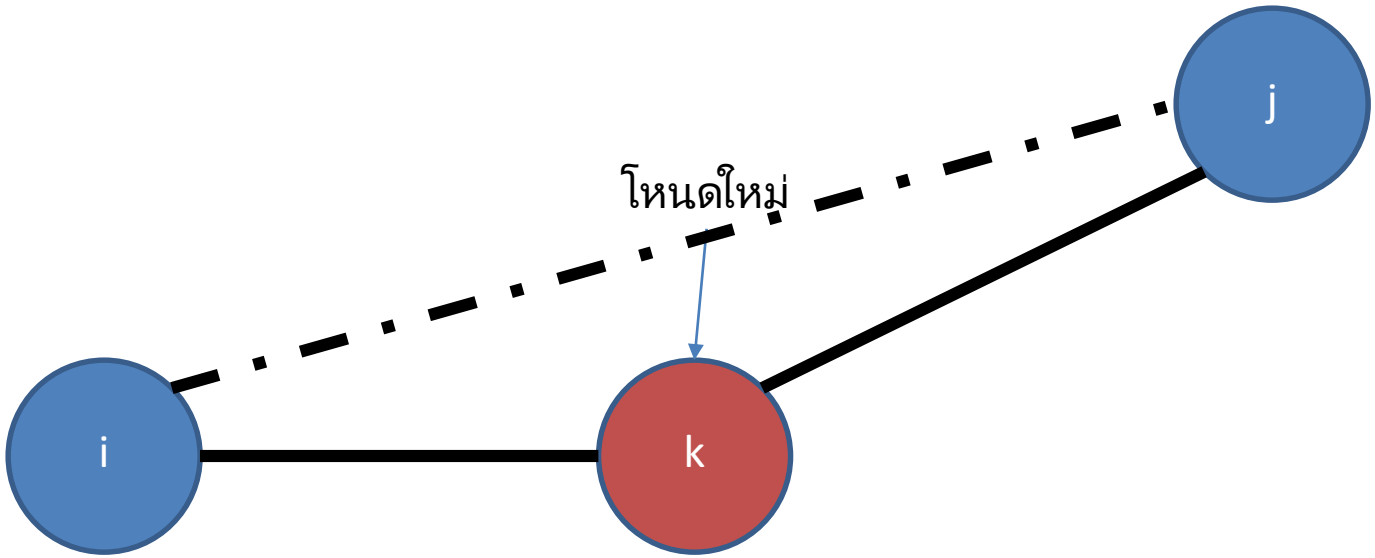


- เริ่มต้นระยะทางจากแต่ละโหนดไปยังตัวมันเองเป็น 0
- และระยะทางจากโหนด a ไป b เป็น x ถ้ามันมีเส้นเชื่อมเชื่อมกันระหว่างโหนด a และ b ด้วย weight x ที่เหลือเป็น infinity

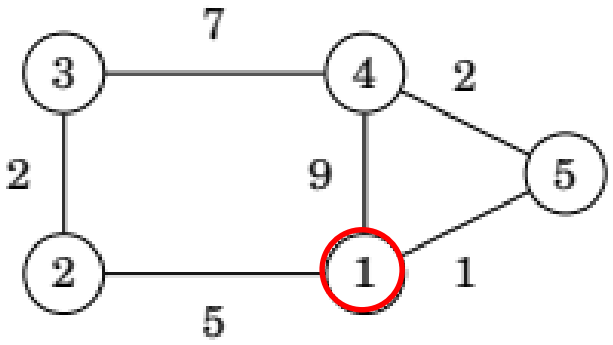
	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	∞	∞
3	∞	2	0	7	∞
4	9	∞	7	0	2
5	1	∞	∞	2	0

การทำงาน

- อัลกอริทึมทำงานเป็นรอบ
- แต่ละรอบอัลกอริทึมจะเลือกโหนดใหม่ k มาเป็นตัวแทน
  - แล้วลองทุกคู่โหนด (i, j) โดยเปรียบเทียบระยะทางเดิม (i, j) กับระยะทางใหม่ที่เป็นระยะทางจาก i ไป k และต่อด้วย k ไป j นั่นคือ (i,k)+(k,j)



- ในรอบแรกเลือก โหนด 1 เป็น intermediate node ใหม่



- เราจะพิจารณาว่า 1 ไปเป็นตัวกลางของ**ทุกคู่**(i,j) แล้วระยะทางลดไหม

```
AdjMatrix[i][j] =
min(AdjMatrix[i][j],AdjMatrix[i][1]+AdjMatrix[1][j])
```

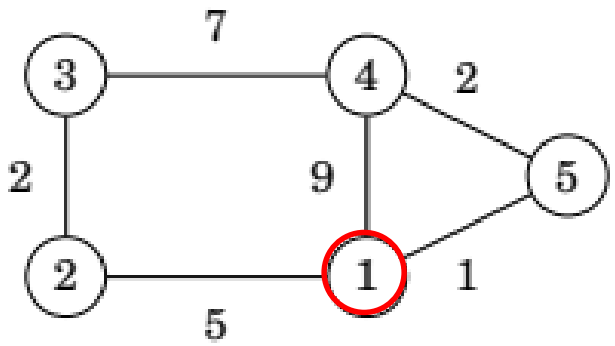
- ตัวอย่างการคำนวณเช่น

- AdjMatrix[2][4] =
min(AdjMatrix[2][4],AdjMatrix[2][1]+AdjMatrix[1][4])

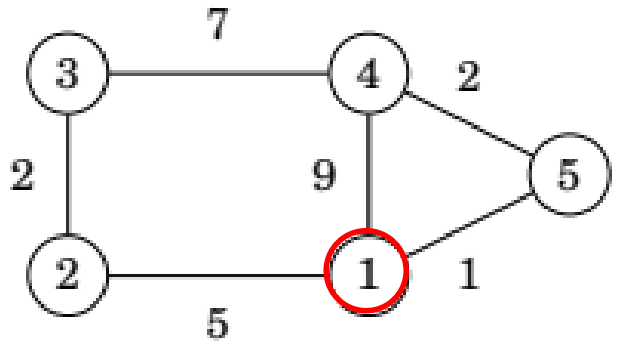
= min(inf,5+9) = 14

- AdjMatrix[3][1] =
min(AdjMatrix[3][1],AdjMatrix[3][1]+AdjMatrix[1][1])
= min(inf, inf) = inf

	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	∞	∞
3	∞	2	0	7	∞
4	9	∞	7	0	2
5	1	∞	∞	2	0



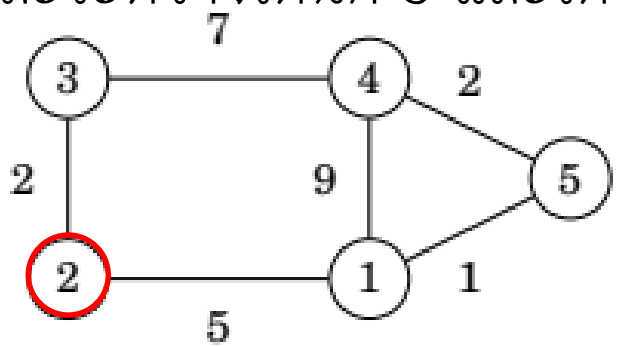
มันคือการถามว่าจากเดิม x ไป y มากกว่า x ไป z แล้ว z ไป y ไหม โดยที่ z =1 นั่นเอง เมื่อทำทุกคู่แล้วจะได้



	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	∞	∞
3	∞	2	0	7	∞
4	9	∞	7	0	2
5	1	∞	∞	2	0

	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	**14**	**6**
3	∞	2	0	7	∞
4	9	**14**	7	0	2
5	1	**6**	∞	2	0

- ในรอบที่สอง โหนด 2 เป็นโหนด intermediate node ใหม่
- เมื่อทำทุกคู่แล้วพบว่า
- สร้าง path ระหว่างโหนด 1 และโหนด 3
- และระหว่างโหนด 3 และโหนด 5

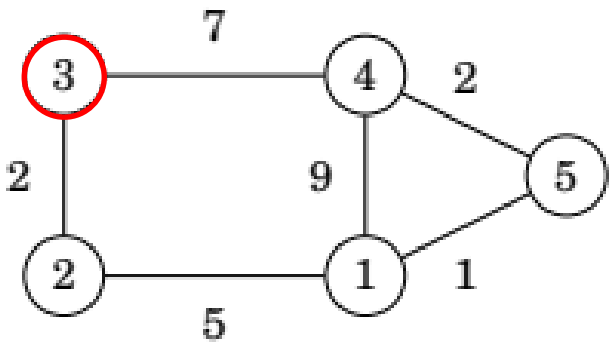


	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	**14**	**6**
3	∞	2	0	7	∞
4	9	**14**	7	0	2
5	1	**6**	∞	2	0

	1	2	3	4	5
1	0	5	**7**	9	1
2	5	0	2	14	6
3	**7**	2	0	7	**8**
4	9	14	7	0	2
5	1	6	**8**	2	0

```
AdjMatrix[3][5]
= min(AdjMatrix[3][5],AdjMatrix[3][2]+AdjMatrix[2][5])
= min(inf,2+6) = 8
```

- ในรอบที่สาม โหนด 3 เป็นโหนด intermediate node ใหม่
- เมื่อลองทุกคู่โหนดแล้วจะสร้าง path ระหว่างโหนด 2 และโหนด 4



	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	**9**	6
3	7	2	0	7	8
4	9	**9**	7	0	2
5	1	6	8	2	0

- อัลกอริทึมจะทำงานต่อไปจน**ทุกโหนด**ได้เป็น intermediate node หลังจากนั้นจะจบการทำงานและ array จะเก็บระยะทางต่ำสุดระหว่างคู่โหนดใดๆ

	1	2	3	4	5
1	0	5	7	3	1
2	5	0	2	8	6
3	7	2	0	7	8
4	3	8	7	0	2
5	1	6	8	2	0

- ตัวอย่างนี้ array บอกว่าระยะทางสั้นสุดจากโหนด 2 ไป 4 คือ 8

Implementation

```
#include <algorithm>
#include <cstdio>
using namespace std;
#define INF 1000000000
int main() {
  int V, E, u, v, w, AdjMatrix[200][200];
  scanf("%d %d", &V, &E);
  for (int i = 0; i < V; i++) {
    for (int j = 0; j < V; j++)
      AdjMatrix[i][j] = INF;
    AdjMatrix[i][i] = 0;
  }

  for (int i = 0; i < E; i++) {
    scanf("%d %d %d", &u, &v, &w);
    AdjMatrix[u][v] = w; // directed graph
  }
```

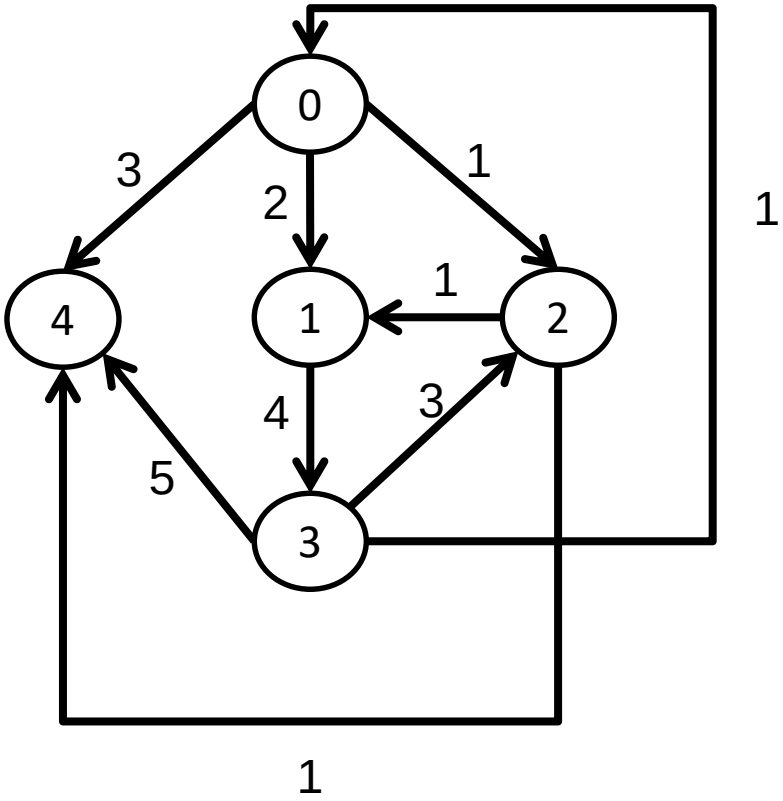
```
  for (int k = 0; k < V; k++)//หยิบทุกโหนดมาเป็ฯ intermediate node
    for (int i = 0; i < V; i++) //update ทุกคู่
      for (int j = 0; j < V; j++)
        AdjMatrix[i][j] = min(AdjMatrix[i][j],AdjMatrix[i][k]+AdjMatrix[k][j]);

  for (int i = 0; i < V; i++)
    for (int j = 0; j < V; j++)
      printf("APSP(%d, %d) = %d\n", i, j, AdjMatrix[i][j]);

  return 0;
}
```

- ประโยชน์ของ Floyd-warshall คือ implement ง่าย เราใช้ matrix AdjMatrix[i][j] เก็บค่า shortest path ระหว่างโหนด a และ b
- เวลาในการทำงานเป็น **$O(n^3)$** เพราะว่ามี 3 loop ซ้อนกัน

- เนื่องจากการ implement Floyd-Warshall algorithm นั้นง่าย จึงเป็นทางเลือกที่ดีในการเริ่มแม้ว่าเราจะต้องการหา single source shortest path
- อย่างไรก็ตาม algorithm นี้ถูกใช้เมื่อกราฟมีขนาดเล็กที่ **$O(n^3)$** ทำงานทัน


0
1
2
3
4
3
2
1
1
1
4
3
5
1